

Memory Efficient Doubly Linked List –insertAfterElement and Time Complexity – Part 3

```
while (curr != nullptr && curr->data != element)
{
    next = XOR(prev, curr->npx);
    prev = curr;
    curr = next;
}
```

When Curr = NULLPTR, when Element is not found in the list.

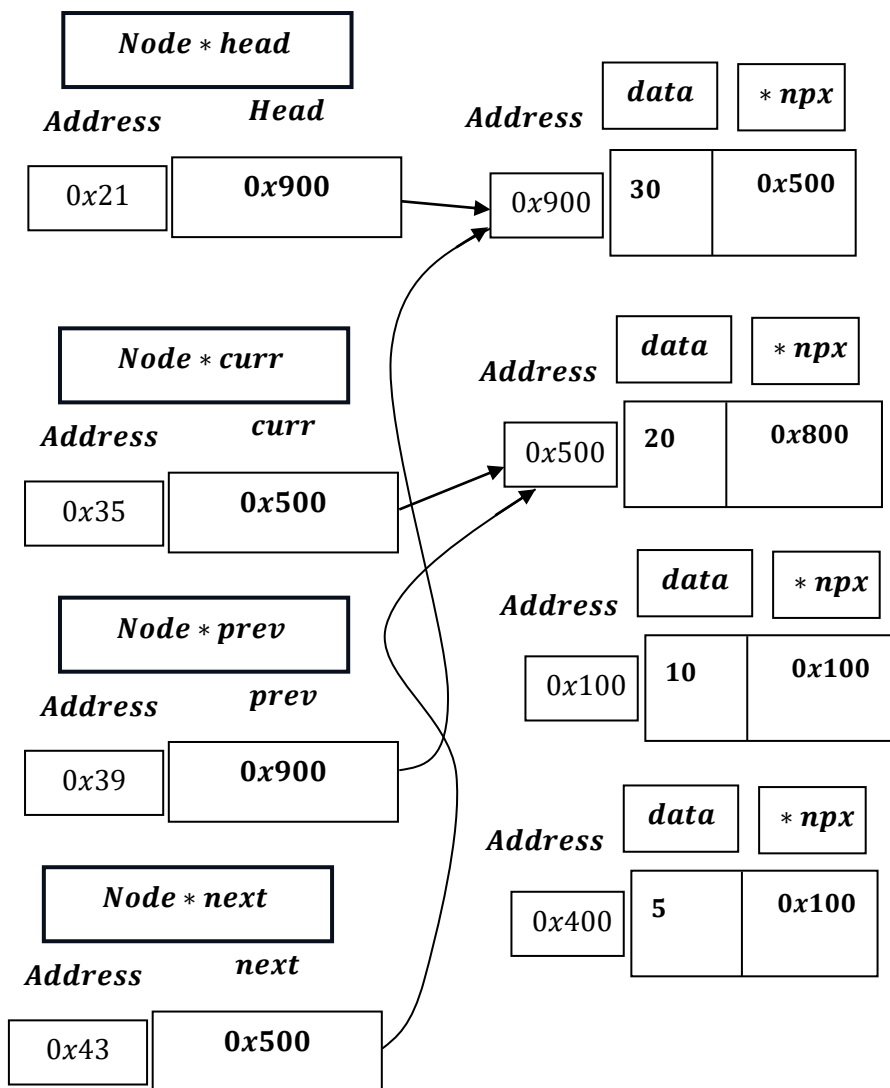
Lets say, Element = 14 and data = 15;

curr: 0x900 $\neq$ nullptr: 0x000 AND curr $\rightarrow$ data = 30 $\neq$ element = 14 :

next = XOR(prev: 0x000, curr $\rightarrow$ npx: 0x500) = 0x000 $\oplus$ 0x500 = 0x500.

prev = curr = 0x900

curr = next = 0x500

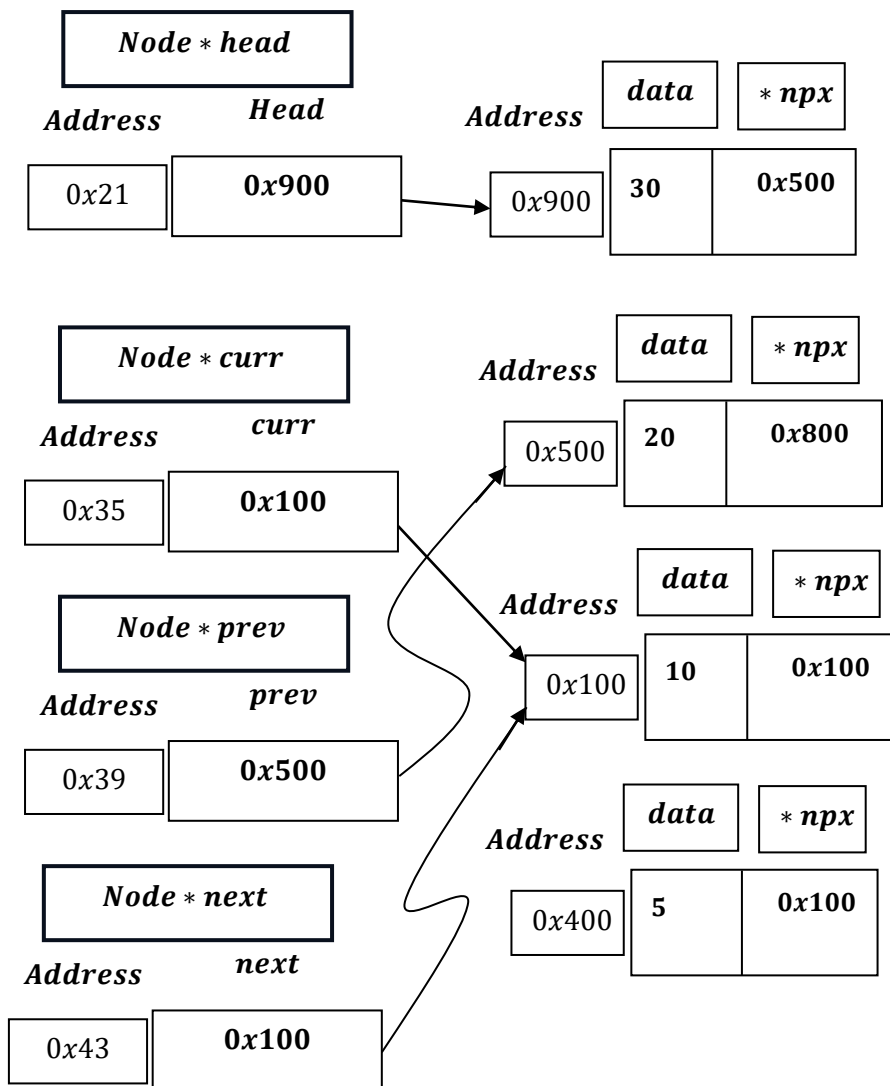


curr: 0x500 ≠ nullptr: 0x000 AND curr → data = 20 ≠ element = 14 :

next = XOR(prev: 0x900, curr → npx: 0x800) = 1001₂ ⊕ 1000₂ = 0001₂ = 0x100

prev = curr = 0x500

curr = next = 0x100

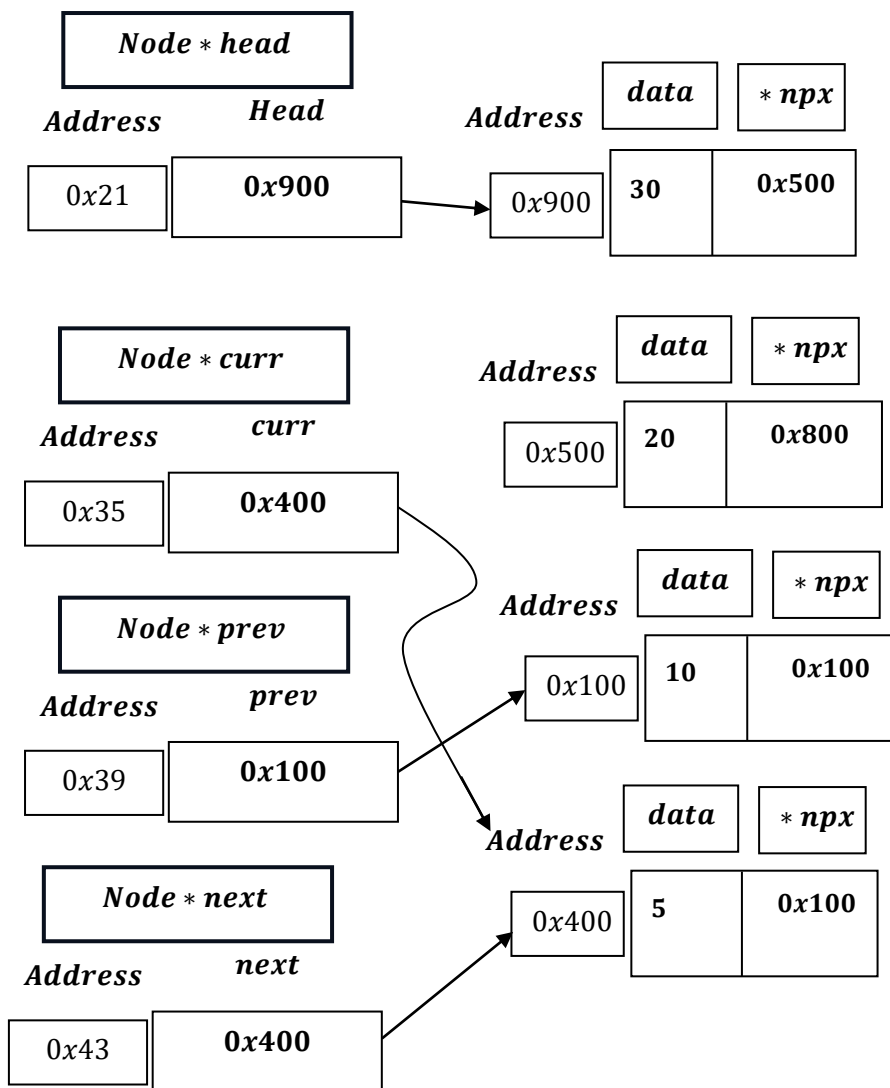


$curr: 0x100 \neq nullptr: 0x000$ AND $curr \rightarrow data = 10 \neq element = 14$:

$next = XOR(prev: 0x500, curr \rightarrow npx: 0x100) = 0101_2 \oplus 0001_2 = 0100_2 = 0x400$

$prev = curr = 0x100$

$curr = next = 0x400$

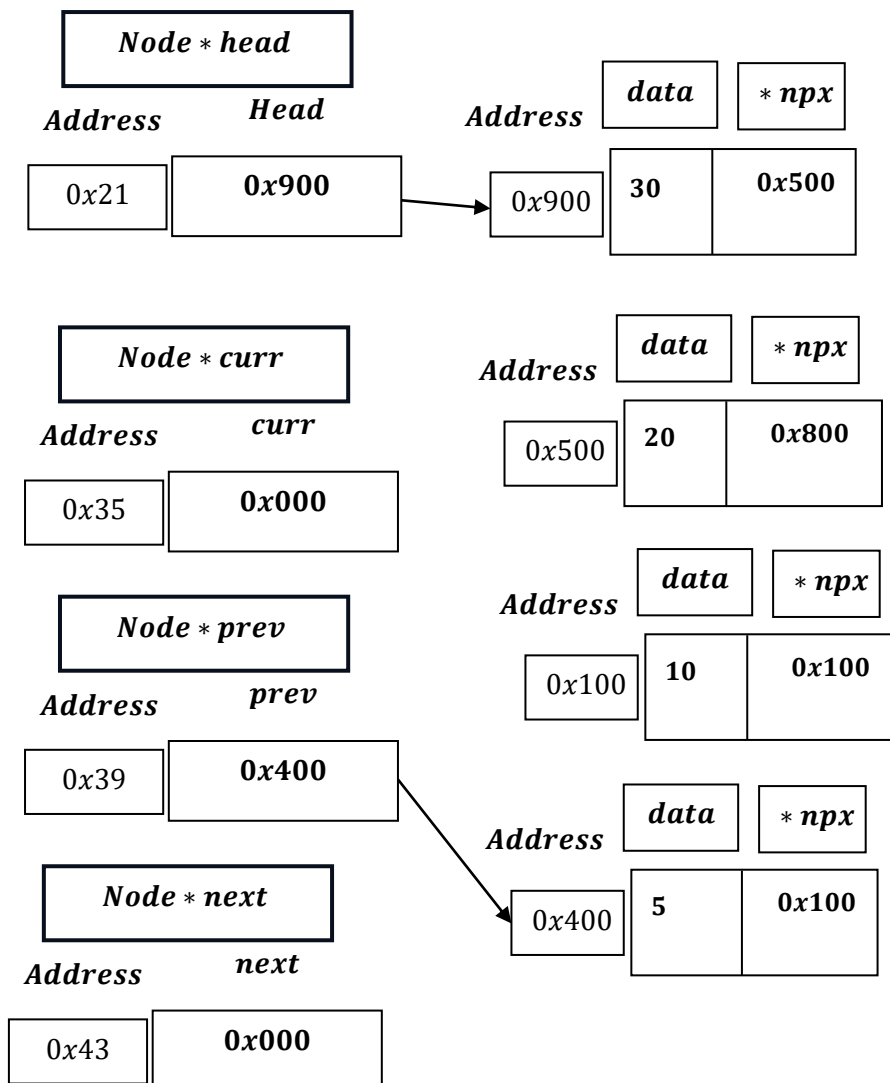


curr: 0x400 $\neq$ nullptr: 0x000[True] AND curr $\rightarrow$ data = 5 $\neq$ element = 14 :

next = XOR(prev: 0x100, curr $\rightarrow$ npx: 0x100) = $0001_2 \oplus 0001_2 = 0000_2 = 0x000$

prev = curr = 0x400

curr = next = 0x000



curr: 0x000 $\neq$ nullptr: 0x000[False] hence ,condition is false and loop exits.

```
if (curr == nullptr)
{
    cout << "Element " << element << " not found in the
list." << endl;
    return;
}
```

As, $curr = nullptr = 0x000$:

Prints: Element 14 not found in the list.

Returns statement return void and exits from the function.

Time Complexity

```
void insertAfterElement(int data, int element)
{
    Node *curr = head;  $\longrightarrow O(1)$ 
    Node *prev = nullptr;  $\longrightarrow O(1)$ 
    Node *next;  $\longrightarrow O(1)$ 

    // Search for element
    while (curr != nullptr && curr->data != element)
    {
        next = XOR(prev, curr->npx);  $\longrightarrow$  condition checks `n + 1` times at Worst Case.
        prev = curr;
        curr = next;
    }

    if (curr == nullptr)  $\longrightarrow O(1)$ 
    {
        cout << "Element " << element << " not found in the list." << endl;
        return;  $\longrightarrow$  Each run `n` times = 3n
    }

    next = XOR(prev, curr->npx);  $\longrightarrow O(1)$ 

    Node *newNode = (Node *)malloc(sizeof(Node));  $\longrightarrow O(1)$ 
    newNode->data = data;  $\longrightarrow O(1)$ 
```

```

// Update pointers using XOR logic
newNode->npx = XOR(curr, next);
curr->npx = XOR(prev, newNode);

if (next != nullptr)
{
    Node *nextNext = XOR(curr, next->npx);
    next->npx = XOR(newNode, nextNext);
}

cout << data << " inserted after " << element << "." << endl;
}

```

Complexity analysis for the code above:

- `newNode->npx = XOR(curr, next);` → $O(1)$
- `curr->npx = XOR(prev, newNode);` → $O(1)$
- The `if` block contains two XOR operations, each taking $O(1)$ time.
- `cout << data << " inserted after " << element << "." << endl;` → $O(1)$

Best Case Analysis

The target `element` is present in the **first node (head)**. The `while` loop condition fails immediately.

Hence complexity : $\Omega(1)$.

Worst Case Analysis

*Just if we focus on the loop ,Worst case occurs when
`Element not Found situation` .*

Loop condition checks $n + 1$ times including when it becomes false, the innerstatement runs:

$next = XOR(prev, curr \rightarrow npx); \Rightarrow 1 + 1 + \dots n \text{ times} = n.$

$prev = curr; \Rightarrow 1 + 1 + \dots n \text{ times} = n.$

$curr = next; \Rightarrow 1 + 1 + \dots n \text{ times} = n.$

Hence ,time complexity at worst complexity: $n + n + n = 3n = 3 \times O(n) = O(n).$

Average Case

Inserted position from 1 to $N + 1$, while loop runs from 1 to $N - 1$.

And when the position is uniformly distributed from 1 to $(N + 1)$:

(Average Case i. e. Undetermined Position)

→ If the Element is inserted after 1st node, the loop's inner statement runs 0 time.

→ If the Element is inserted after 2nd node, the loop's inner statement runs 1 time.

→ If the Element is inserted after 3rd node, the loop's inner statement runs 2 time.

→ If the Element is inserted after 4th node, the loop's inner statement runs 3 time.

.

.

→ If the Element is inserted at $(n + 1)^{th}$ node, the loop's inner statement runs $N - 1$ time.

$$\therefore, \sum_{i=0}^{N-1} i = \frac{N(N-1)}{2}$$

Now, If there is 'N' no. of nodes there we can insert data at 2 to $N + 1$ positions or repositioning 1 to N .

For position 1, the probability of insertion is $\frac{1}{N}$.

For position 2, the probability of insertion is $\frac{1}{N}$.

.

.

For position $N + 1$, the probability of insertion is $\frac{1}{N}$.

Hence probability of insertion will always remain $\frac{1}{N}$.

$\therefore$ Average Time Complexity is : probability of insertion $\times$ (number of possible iterations)

$$= \frac{1}{N} \times \left(\sum_{i=0}^{N-1} i \right)$$

$$= \frac{1}{N} \times \frac{N(N-1)}{2}$$

$$= \frac{N-1}{2}$$

Applying , Big O on the above equation:

$$= \frac{1}{2} \times O(N) - O\left(\frac{1}{2}\right)$$

$$= O(N)$$

Hence, Average Time Complexity is : $\Theta(N)$.

Note , by relation: $\Omega \leq \Theta \leq O$, we get:

$$\Rightarrow c_1 \times g(n) \leq f(n) \leq c_2 \times g(n)$$

$$\Rightarrow c_1 \times 1 \leq \frac{N-1}{2} \leq c_2 \times N$$

***Average Case Time Complexity[Most Simplest Alternative Apporach]:
(Simplest approach)***

***(If the elements in the linked list are uniformly distributed
and the key is equally likely to be any element in the list)***

***Note: –The checking that Element or Key is found or not , done in the
condtion of while loop, and the condition checks `n` times .***

–if key is found at 1st Node , Key/Element searched for 1 time .

–if key is found at 2nd Node , Key/Element searched for 2 time .

–if key is found at 3rd Node, Key/Element searched for 3 time .

.

.

–if key is found at Nth Node , Key/Element searched for N time .

As , the key is equally likely to be any element in the list,

Probablity of finding 1st element (p_1), if key equals to 1st element = $\frac{1}{N}$.

Probablity of finding 2nd element (p_2), if key equals to 2nd element = $\frac{1}{N}$.

.

.

Probability of finding N^{th} element (p_2), if key equals to N^{th} element $= \frac{1}{N}$.

Therefore,

$$\text{Probability}(P) = p_1 = p_2 = \dots = p_n = \frac{1}{N}.$$

Therefore, Average Case = Number of times the Key is searched $\times$ probability of finding elements.

That is the average number of executions is the sum of all possible executions weighted by their probabilities. This is given by:

$$\begin{aligned} \therefore \text{Average Case} &= \frac{1}{N} \times \left(\sum_{i=1}^N i \right) \\ &= \frac{1}{N} \times \frac{N(N+1)}{2} \left[\sum_{i=1}^N i = \frac{N(N+1)}{2} \right] \end{aligned}$$

$$= \frac{N+1}{2}$$

$$= \frac{N}{2} + \frac{1}{2}$$

Applying , Big Theta Θ we get :

$$= \frac{1}{2} \times (\Theta(N) + \Theta(1))$$

$$= \Theta(N) + \Theta(1)$$

$$= \Theta(N).$$

Note , by relation: $\Omega \leq \Theta \leq O$, we get:

$$\Rightarrow c_1 \times g(n) \leq f(n) \leq c_2 \times g(n)$$

$$\Rightarrow c_1 \times 1 \leq \frac{N+1}{2} \leq c_2 \times N$$

<i>Memory Efficient Doubly Linked List</i>	<i>Cases</i>	<i>Time Complexity</i>
<i>Insert After Element</i>	<i>1. Best Case</i>	$\Omega(1)$
	<i>2. Worst Case</i>	$O(n)$
	<i>3. Average Case</i>	$\Theta(n)$
